# Building a Scalable, Secured, and Highly Available Cloud Application

Event Ticket Booking Platform

An assignment report submitted to the

Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

In partial fulfillment of the requirements for

**EC7205 — Cloud Computing**

| | | |
|---|---|---|
| Achintha H.G.R. | – | EG/2021/4384 |
| Horadagoda H.A.D.A.M | – | EG/2021/4557 |
| Munasinghe B.P.R.D | – | EG/2021/4680 |
| Pathirathna A.H. | – | EG/2021/4705 |

# 1 Introduction

This report describes the architecture and design of the **Event Ticket Booking Platform**, which has been developed throughout the EC7205 Cloud Computing course at the University of Ruhuna. The Event Ticket Booking Platform is a fully functional cloud-based application and highlights some of the basic principles of cloud computing such as microservices, container orchestration, automated CI/CD pipelines and cloud native security practices.

The application allows the user to search for events, buy tickets, and get their bookings confirmed, whereas the administrator gets access to all the features required for managing the event from start to finish. The software is hosted on the Google Kubernetes Engine(GKE) and incorporates several cloud technologies such as Amazon Simple Storage Service (AWS S3), Keycloak, RabbitMQ, and Brevo.

The topics covered under this report are the design and architecture, scope and requirements of the project, security architecture, deployment architecture on cloud, message handling, CI/CD pipeline, and state flow.

# 2 Design Approach & Architecture

## 2.1 Chosen Architecture

This is implemented using the **Microservice Architecture** running on the Google Kubernetes Engine (GKE). The microservices architecture was chosen as opposed to the monolithic approach due to the following key factors: **Scalability of Individual Services**, **Isolation of Faults** (a notification failure will not crash the booking and event services) and **Use of Different Technologies**.

## 2.2 Architectural Patterns Applied

| **Pattern** | **Application in this project** |
|---|---|
| API Gateway | Spring Cloud Gateway — single entry point; routing and JWT policy enforcement |
| Service Discovery | Eureka Server for Spring service registration and resolution |
| Externalised Config | Spring Cloud Config Server reading from dedicated `config-repo` Git repository |
| Async Messaging | RabbitMQ decouples notification delivery from core booking/auth transactions |
| CQRS (partial) | Public read endpoints unauthenticated; write/admin endpoints require role-bearing JWT |
| Sidecar Config | Kubernetes ConfigMap and Secret inject environment-specific values at pod startup |

### 2.3 Technology Stack

| Layer | Technology | Reason |
|---|---|---|
| Backend services | Spring Boot 3 | Mature ecosystem, Spring Security JWT, JPA/MySQL integration |
| Notification svc | Express.js + Node | Lightweight, ideal for queue-driven async processing |
| User frontend | Next.js (React) | SSR for SEO, fast page loads for event discovery |
| Admin frontend | Angular | Structured framework for data-heavy dashboards |
| API Gateway | Spring Cloud Gateway | Native Spring integration, route predicates, path rewriting |
| Identity | Keycloak | Production-grade OAuth2/OIDC, realm-based role management |
| Databases | MySQL + MongoDB | MySQL for ACID transactions; MongoDB for notification logs |
| Object storage | AWS S3 | Scalable image and file storage for avatars and event images |
| Message broker | RabbitMQ | Reliable topic-based async messaging with persistence |
| Orchestration | Kubernetes / GKE | Container orchestration, self-healing, declarative deployment |
| CI/CD | GitHub Actions | Per-service pipelines with SonarQube quality gate |

## 3 Project Scope & Requirements

### 3.1 In Scope

We aim to include user registration, OTP email verification, login, password reset, and profile avatar upload along with asynchronous transactional email notifications via RabbitMQ and Brevo. We will enable Role-based access control with three roles.

### 3.2 Key Requirements Summary

**Functional Requirements** are Authentication with OTP, Event and booking management, email notifications, RBAC with three roles

**Scalability Requirements** are Stateless microservices independently scalable via Kubernetes replica adjustment.

**Security Requirements** are JWT validation at Gateway and at each service, Spring Security role enforcement, secrets injected via Kubernetes Secrets but never stored in source code.

**Reliability Requirements** are Kubernetes self-healing, RabbitMQ with PVC for message durability, notification decoupling prevents email failures from blocking core transactions.

**Maintainability Requirements** are Per-service CI/CD pipelines with quality gates and externalised config via Spring Cloud Config Server allowing environment changes without image rebuilds.

# 4 Security Architecture

## 4.1 Role-Based Access Control

| Role | Permissions | Restricted from |
|---|---|---|
| `admin` | Full access: manage categories, events, users, bookings, notifications | N/A |
| `host` | Create and manage own events; view own bookings | User management, other hosts' events |
| `user` | Browse events, create bookings, manage own profile | Event creation, admin routes |
| *unauthenticated* | Read public event listings and categories only | All write operations and bookings |

## 4.2 Layered Security Model

**In Layer 1 API Gateway,** Spring Cloud Gateway enables specifically listed public paths (event browsing, auth endpoints) without a bearer token. All other requests are needed to carry a valid JWT. The gateway gives validation to token presence and signature yet it does not resolve roles.

**In Layer 2 Downstream services,** each Spring Boot service is configured as a Spring Security OAuth2 resource server. It independently validates the JWT against Keycloak's JWK endpoint and maps the `realm_access.roles` claim to Spring `GrantedAuthority` objects. Method-level `@PreAuthorize` annotations enforce role requirements, providing defence in depth beyond gateway-level policy.

**In Secret Management,** Sensitive values (database URLs, passwords, Keycloak secrets, S3 keys, Brevo API key) are stored in Kubernetes `Secret` resources (`event-hub-secrets`) and injected as environment variables at pod startup. Non-sensitive configuration uses `ConfigMap` (`event-hub-config`) and Spring Cloud Config Server.

# 5 Cloud Deployment Architecture

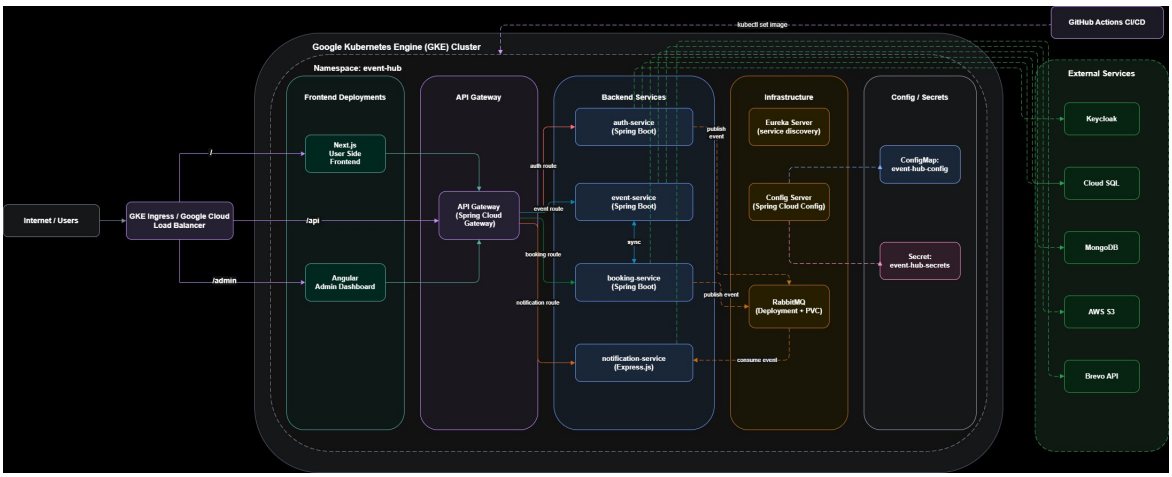


**Figure 1:** Cloud Architecture — Event Ticket Booking Platform on GKE

The deployment targets a GKE cluster with all workloads in the `event-hub` namespace. External traffic enters through a GKE Ingress / Cloud Load Balancer, routing `/api` to the Spring Cloud Gateway and `/admin` to the Angular admin dashboard. The Gateway forwards to four backend services: `auth-service`, `event-service`, `booking-service`, and `notification-service`. Infrastructure services include Eureka (service discovery), Config

Server, and RabbitMQ with a PersistentVolumeClaim for message durability. External dependencies — Keycloak, Cloud SQL/MySQL, MongoDB, AWS S3, and Brevo — are consumed from within the cluster. GitHub Actions CI/CD performs rolling updates via `kubectl set image` on each push to `main`.

## 6 Messaging Architecture

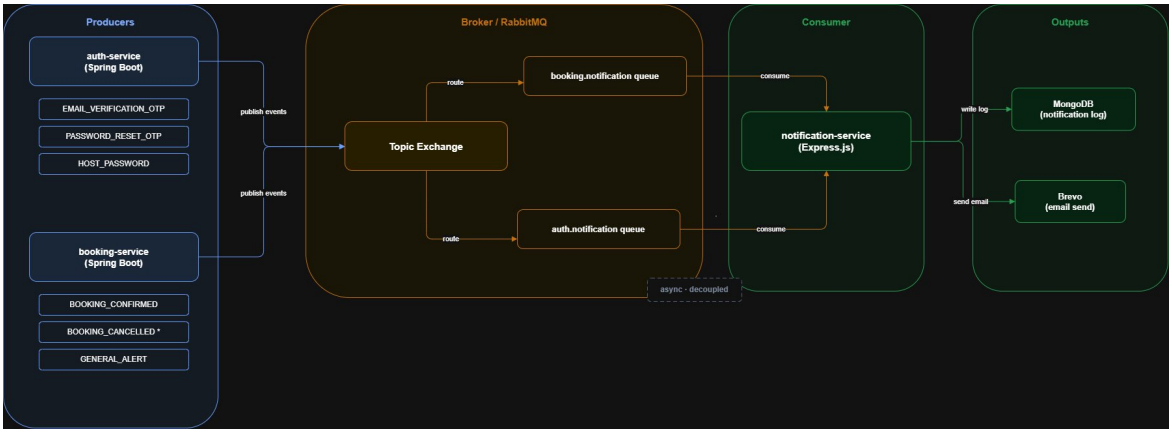


**Figure 2:** Messaging Architecture — RabbitMQ Async Event Flow

RabbitMQ implements a publish/subscribe pattern via a **Topic Exchange** routing events to two dedicated queues. `auth-service` publishes `EMAIL_VERIFICATION_OTP`, `PASSWORD_RESET_OTP`, and `HOST_PASSWORD` events; `booking-service` publishes `BOOKING_CONFIRMED` and `GENERAL_ALERT`. The `notification-service` (Express.js, `amqplib`) consumes both queues, renders the appropriate email template, dispatches via Brevo, and writes an audit record to MongoDB. Producers return HTTP responses immediately after publishing — email delivery is fully decoupled from the transaction path.

## 7 CI/CD Pipeline

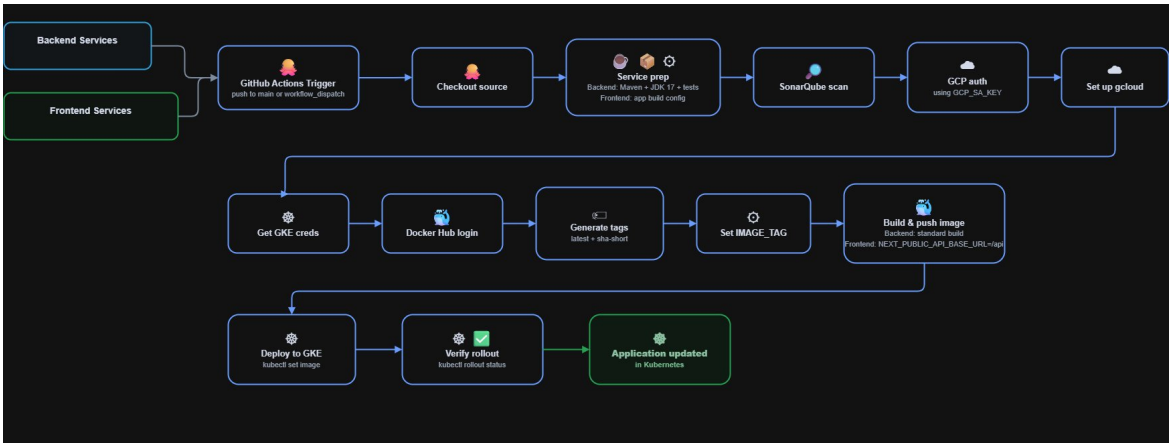


**Figure 3:** CI/CD Pipeline — Applied to All Service Repositories

Each service repository has a GitHub Actions workflow triggered on push to `main`. Pipeline stages: **(1)** checkout source; **(2)** Maven/JDK 17 build with unit tests (backend) or Node.js dependency install and build (frontend); **(3)** SonarQube quality gate scan — a failing scan blocks all downstream stages; **(4)** GCP authentication and GKE credential fetch; **(5)** Docker Hub login; **(6)** build and push Docker image with `latest` and short-SHA tags; **(7)** `kubectl set image` rolling update on the target Deployment in `event-hub` namespace; **(8)** `kubectl rollout status` confirms successful rollout before the pipeline exits.

# 8 State Diagrams

## 8.1 Authentication Flows

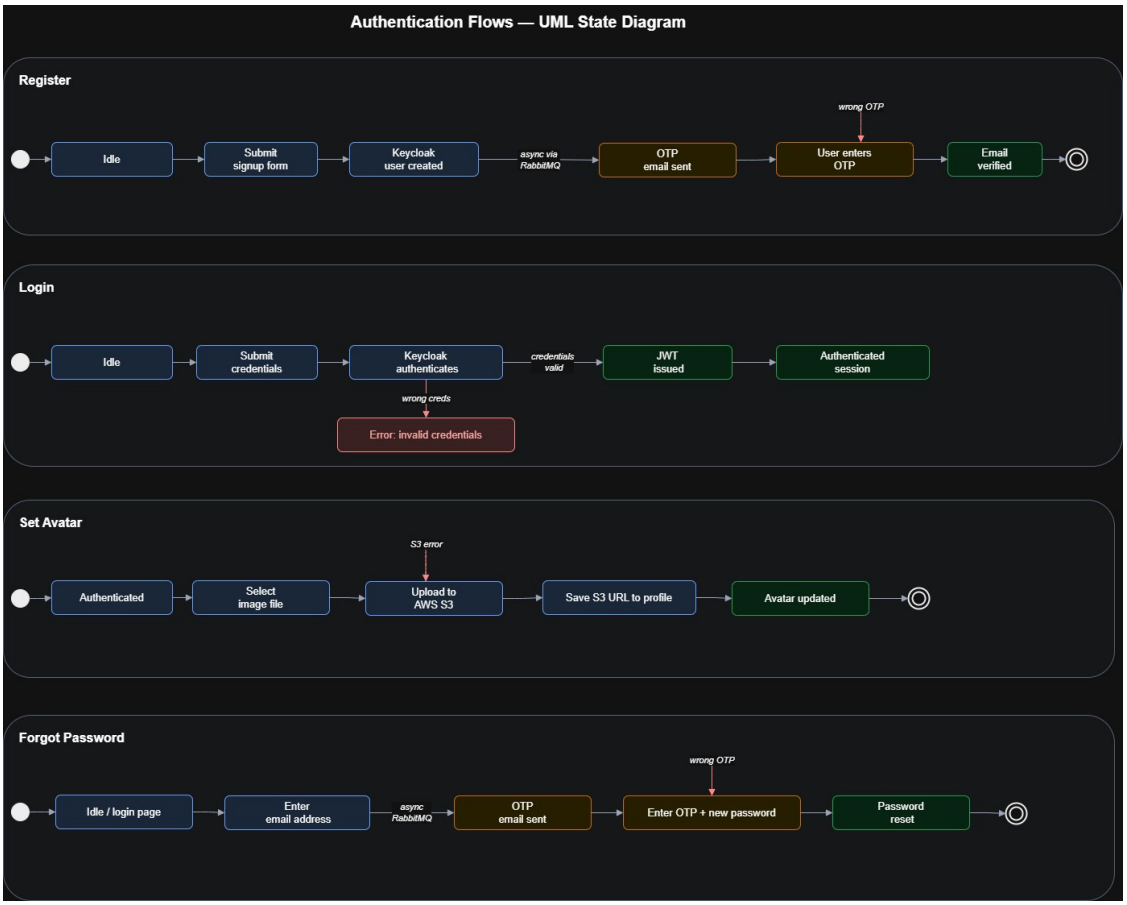


**Figure 4:** State Diagram — Authentication Flows (Login, Register, Set Avatar, Forgot Password)

**Register:** Form submitted → Keycloak identity + MySQL profile created → async OTP published to RabbitMQ (201 returned immediately) → user verifies OTP to activate. **Login:** Credentials delegated to Keycloak → JWT + refresh token issued on success. **Set Avatar:** Image uploaded to AWS S3 → S3 URL saved to MySQL profile. **Forgot Password:** Async `PASSWORD_RESET_OTP` published → correct OTP + new password updates Keycloak.

## 8.2 Category and Event Creation

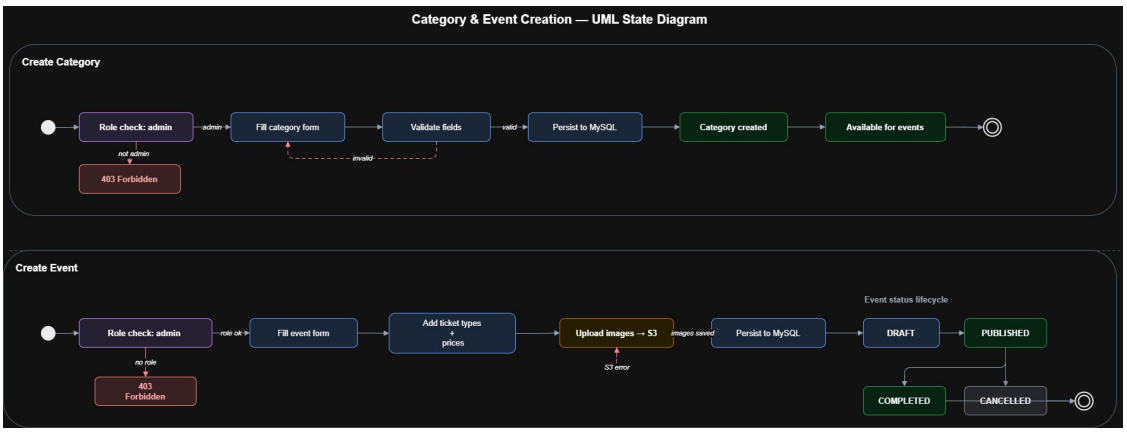


**Figure 5:** State Diagram — Create Category and Event

**Create Category:** `admin` role required (403 otherwise) → server-side validation → persisted to MySQL. **Create Event:** `admin`/`host` role required → form with ticket types, pricing, venue → images uploaded to S3 → persisted as `DRAFT`. Lifecycle: `DRAFT` → `PUBLISHED` →

`COMPLETED`/`CANCELLED`. Only published events visible to users.

### 8.3 Booking an Event

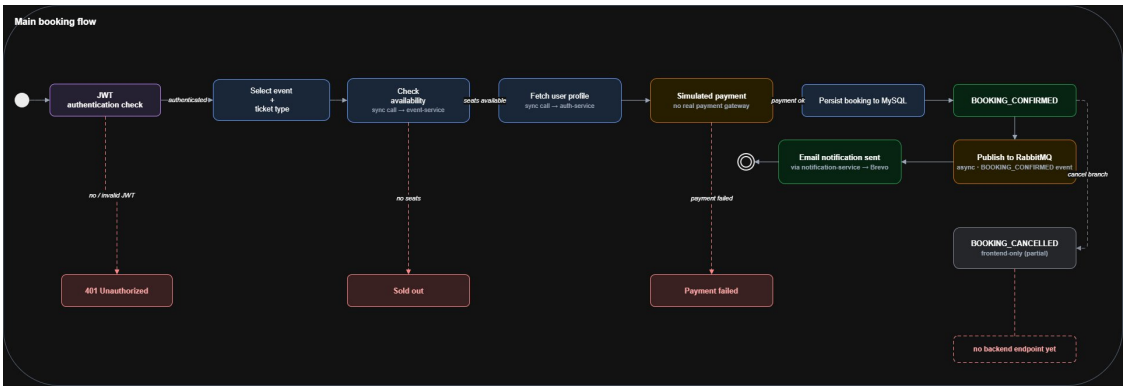


**Figure 6:** State Diagram — Booking an Event

**(1)** JWT verified at Gateway and `booking-service` (401 on failure). **(2)** User selects event, ticket type, quantity. **(3)** Sync REST call to `event-service` checks availability (*Sold out* if none). **(4)** Sync REST call to `auth-service` fetches user profile. **(5)** Payment simulated internally (*Payment failed* on error). **(6)** Booking persisted to MySQL as `BOOKING_CONFIRMED`. **(7)** `BOOKING_CONFIRMED` event published to RabbitMQ; HTTP 201 returned immediately. `notification-service` consumes the event, dispatches email via Brevo, logs to MongoDB. Cancellation frontend call exists but backend endpoint is not yet complete.